

Debugging R

Hanne Oberman 

Junior assistant professor at Utrecht University

28 Sep 2026



Attribution and reuse



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/) (CC BY-SA4.0).

These materials have been developed in part by [Hanne Oberman](#), [Shannon Pileggi](#), and [Jon Harmon](#).

When using these materials, you *must* ensure that the content is not used for generative AI model training.



Computer bugs

92

9/9

0800 Antan started
 1000 " stopped - antan ✓

1300 (032) MP-MC { 1.2700 9.037 847 025
 (033) PRO 2 2.130476415 9.037 846 985 correct
 correct 2.130676415 4.615925059(-2)

Relays 6-2 in 033 failed special speed test
 in relay room test.

Relays changed

1700 Started Cosine Tape (Sine check)
 1525 Started Multi Adder Test.

1545  Relay #70 Panel F
 (moth) in relay.

First actual case of bug being found.

1630 Antan started.
 1700 closed down.

Relay 2145
 Relay 3370



Bugs in R

```
> df$a
```

```
Error in df$a : object of type 'closure' is not subsettable
```

Explanation: `df` is a closure, in this case, the function `stats::df()`, the density of F -distribution.

(It is not a data frame, as you thought it was, ha!)

It does not have elements that you could subset.



Types of errors

- Parsing errors: you are missing a closing `)`, `}` or `"` so the code `AST` cannot even be formed
 - Usually captured by `IDEs` such as RStudio and Positron with red flashing pointers
- Syntax errors:
 - Parentheses are matched but one of them closes at wrong time
 - The name of a function input is misspelled
 - Passing more inputs to a function than it expects
- Runtime errors: the code is syntactically correct, but it does a weird thing in your actual environment with your actual data



Common syntax errors

Longer list here

- Parenthesis mismatches
- `[[...]]` vs. `[...]`
- `==` vs. `=`
- Comparing real numbers exactly using `==` after calculations that result in non-integers
- Type coercion and dropping dimensions

```
1 inherits(mtcars[,1], "data.frame")
```

```
[1] FALSE
```

```
1 inherits(mtcars[,1, drop=FALSE], "data.frame")
```

```
[1] TRUE
```



More common syntax errors

- You expect a single value but your code gives you a vector

```
> if (c(1,2)) { }
```

```
Error in if (c(1, 2)) { : the condition has length > 1
```

→ you may need `identical()` or `all()`, or form the condition more thoroughly



Learning objectives:

- Overall: Find and fix errors.
- Locate exactly where an error occurred with the `traceback()` function.
- Pause the execution of a function and launch an environment to interactively explore what's happening.
- Debug non-interactively executed code.
- Recognize other (non-error) problems that occasionally also need debugging.



Strategies for finding and fixing errors

Debugging is like being the detective in a crime movie where you're also the murderer.

-Filipe Fortes



Troubleshooting

1. SEARCH

Teach Me How To Google, Samantha Csik (2022)
[slides](#), [recording](#)

2. RESET

Restart R, especially when things get weird.

3. REPREX

make a reprex..... please, Sharla Gelfand (2021)
[slides](#), [recording](#), <https://reprex.tidyverse.org/>

4. AI (optional)

Claude, ChatGPT, etc.



1. Search first!

- Read the docs (e.g., `??`, or `usethis::browse_package()`)
- Google new error messages to help translate
- Find other instances of the error on the web (e.g., GitHub Issues, `usethis::browse_github()`, or StackOverflow)
- Packages that help:
 - `{errorist}`
 - `{searcher}`
 - Probably AI agents after 2026



2. Resetting R

Have you tried turning it **OFF** and **ON** again?

Session → Restart R or

- Ctrl + Shift + F10 (Windows),
- Cmd + Shift + 0 / Cmd + Shift + F10 (Mac)

`usethis::use_blank_slate()`

or Tools → Global Options → Workspace

Workspace

☐ Restore .RData into workspace at startup

Save workspace to .RData on exit: Never ▼



Why reset?

Which persist after `rm(list = ls())`?

Option	Persists?
A. <code>library(dplyr)</code>	
B. <code>summary <- head</code>	
C. <code>options(stringsAsFactors = FALSE)</code>	
D. <code>Sys.setenv(LANGUAGE = "fr")</code>	
E. <code>x <- 1:5</code>	
F. <code>attach(iris)</code>	



Why reset?

Which persist after `rm(list = ls())`?

Option	Persists?
A. <code>library(dplyr)</code>	✓
B. <code>summary <- head</code>	
C. <code>options(stringsAsFactors = FALSE)</code>	✓
D. <code>Sys.setenv(LANGUAGE = "fr")</code>	✓
E. <code>x <- 1:5</code>	
F. <code>attach(iris)</code>	✓



Instead, reset.

And/or use `sessioncheck()`.

```
1 sessioncheck::sessioncheck()
```

Warning: Session check results:

- ✖ Unexpected objects in global environment: fa_fill
- ✖ Unexpected packages: countdown
- ✓ No unexpected environments attached



Your turn, exercise 00

Set your Workspace options as shown.

`usethis::use_blank_slate()` or

Tools → Global Options → Workspace

Workspace

☐ Restore .RData into workspace at startup

Save workspace to .RData on exit: Never ▼



3. Repeatable bugs are more debuggable

- May need to execute many times
- Up-front investment to make a `{reprex}` pays off
- A **reproducible example** is a minimal and self-contained code snippet that shows exactly what happened when the code was run
 - Is the example minimal? Remove unrelated code and data, etc.
 - Is it self-contained? All libraries loaded, seeds set, data available, etc.



The `reprex()` function

- `reprex()` records:
 - the R expressions you pass (or the selected code in RStudio)
 - the console output, messages, warnings, and plots
- The minimal, self-contained snippet speeds up diagnosing problems
- The output can be copied directly into forums, issue trackers, research and teaching materials
- It enforces the reproducibility principle: anyone who receives the snippet can run it on their own machine and obtain the exact same result



A reprex

```
1 A <- 1:3
2 mean(A)
```

```
[1] 2
```

```
1 mean(a)
```

```
Error:
! object 'a' not found
```

```
1 reprex::reprex()
```

```
1 A <- 1:3
2 mean(A)
3 #> [1] 2
4 mean(a)
5 #> Error:
6 #> ! object 'a' not found
```

Created on 2026-09-24 with [reprex v2.1.1](#)



Use the scientific method to identify bugs

- Generate a hypothesis.
- Design experiments to test that hypothesis.
- Record your results.

Being systematic often saves time in the end.



But what if



Debugging in practice

Finding your bug is a process of confirming the many things that you *believe* are true — until you find one which is *not* true.

—Norm Matloff (emphasis added)



Checklist

✓ R installed? Pretty recent?

→ Recommended $R \geq 4.4.0$

✓ RStudio installed?

→ I'm on 2026.08.1+195

✓ Ready to build packages?

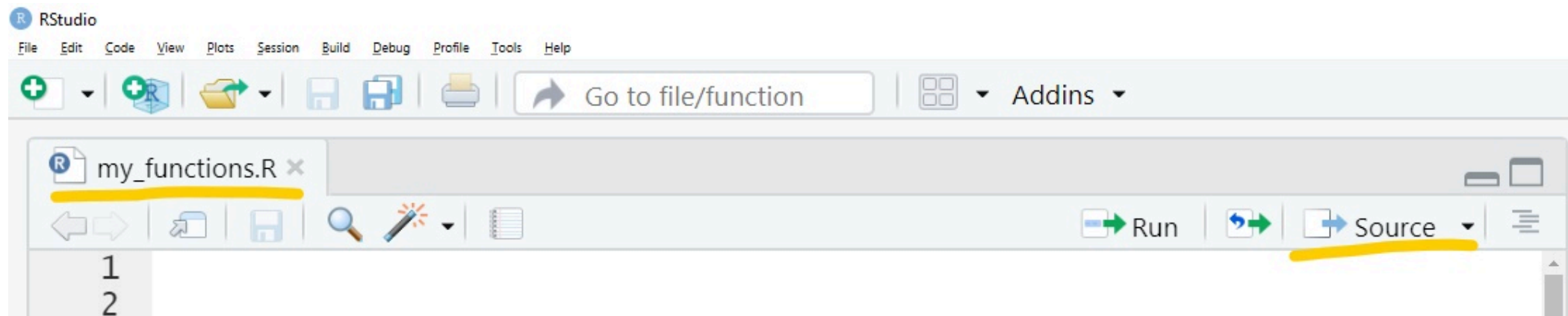
→ `devtools::has_devel()`

→ *Your system is ready to build packages!*



Sourcing

- ⚠ name your script with functions
 - ⚠ source your script with functions
- for the best debugging experience 😎



Key concepts

 traceback

→ **location** where did the error occur

 interactive debugger

→ **context** why did the error occur

 your code vs  their code



Debugging tools

 **traceback**

 **interactive debugger**

 **your code**

 **their code**

Functions

		<code>print() / cat() / message()</code>	output diagnostic information in code
	 	<code>traceback()</code>	locate error
		<code>browser()</code>	open interactive debugger at browser() location
	 	<code>debug() / debugonce()</code>	automatically open interactive debugger when function called
	 	<code>trace()</code>	start debugger at specific location in function

options()

	 	<code>error = rlang::entrace</code>	provides richer traceback & error handling
	 	<code>error = recover</code>	select frame from traceback to enter interactive debugger
	 	<code>warn = 2</code>	upgrade warnings to errors
	 	<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace

IDE

		breakpoint	open interactive debugger at breakpoint location
	 	Debug -> On Error -> Error Inspector	provides "Show Traceback" & "Rerun with Debug" console options
		Debug -> On Error -> Break in code	open interactive debugger on error



This is a lot.

These tools achieve similar objectives slightly differently.

People don't generally use *all* of these tools at once. They pick and choose the ones they like.



Debugging tools

	 traceback	 interactive debugger	 your code	 their code
Functions		<code>print() / cat() / message()</code>	output diagnostic information in code	
	  	<code>traceback()</code>	locate error	
	 	<code>browser()</code>	open interactive debugger at browser() location	
	  	<code>debug() / debugonce()</code>	automatically open interactive debugger when function called	
	  	<code>trace()</code>	start debugger at specific location in function	
options()	  	<code>error = rlang::entrace</code>	provides richer traceback & error handling	
	  	<code>error = recover</code>	select frame from traceback to enter interactive debugger	
	 	<code>warn = 2</code>	upgrade warnings to errors	
	  	<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace	
IDE	 	<code>breakpoint</code>	open interactive debugger at breakpoint location	
	  	<code>Debug -> On Error -> Error Inspector</code>	provides "Show Traceback" & "Rerun with Debug" console options	
	 	<code>Debug -> On Error -> Break in code</code>	open interactive debugger on error	





Set up

```
1 f <- function(x) {  
2   x + 1  
3 }  
4  
5 g <- function(x) f(x)  
6  
7 g("a")
```

Error in `x + 1`:
! non-numeric argument to binary operator



traceback()

```
1 source("demo/my_functions.R")  
2 g("a")  
3 traceback()
```

```
2: f(x) at my_functions.R#5  
1: g("a")
```

shows the sequence of calls that lead to the error.



Richer traceback

```
1 source("demo/my_functions.R")
2 options(error = rlang::entrace)
3 g("a")
4 rlang::last_trace()
```

<error/rlang_error>

Error:

! non-numeric argument to binary operator

Backtrace:

```
1. └─global g("a")
2.   └─global f(x) at wtf-debugging-slides/demo/my_functions.R:5:5
```

`options(error = rlang::entrace)`
could go in your `.Rprofile`



traceback vs rlang functions

Numbering and ordering differs between `traceback()` and rlang functions.

<code>traceback()</code>	<code>where</code>	<code>recover()</code>	rlang functions
5: stop("...")			
4: i(c)	where 1: i(c)	1: f()	1. $\perp_{\text{global}}::f(10)$
3: h(b)	where 2: h(b)	2: g(a)	2. $\perp_{\text{global}}::g(a)$
2: g(a)	where 3: g(a)	3: h(b)	3. $\perp_{\text{global}}::h(b)$
1: f("a")	where 4: f("a")	4: i("a")	4. $\perp_{\text{global}}::i("a")$



Example: Chained functions

```
f <- function(a) g(a)
g <- function(b) h(b)
h <- function(c) i(c)
i <- function(d) {
  if (!is.numeric(d)) {
    stop("`d` must be numeric", call. = FALSE)
  }
  d + 10
}
```

[Copy](#)

```
f("a")
```

Error: `d` must be numeric

[Show Traceback](#)
[Rerun with Debug](#)



`traceback()` shows the call stack that lead to the error

Click “Show traceback”::

```
> f("a")
```

```
Error: `d` must be numeric
```

 Hide Traceback

 Rerun with Debug

```
5. stop("`d` must be numeric", call. = FALSE) at debugging.R#6  
4. i(c) at debugging.R#3  
3. h(b) at debugging.R#2  
2. g(a) at debugging.R#1  
1. f("a")
```

Read bottom to top.



traceback() is confusing with lazy evaluation

```
j <- function() k()  
k <- function() stop("Oops!", call. = FALSE)  
f(j())  
#> Error: Oops!
```

[Copy](#)

```
traceback()  
#> 7: stop("Oops!") at #1  
#> 6: k() at #1  
#> 5: j() at debugging.R#1  
#> 4: i(c) at debugging.R#3  
#> 3: h(b) at debugging.R#2  
#> 2: g(a) at debugging.R#1  
#> 1: f(j())
```

[Copy](#)

traceback() limits

In real world, traceback may look like 25 layers

- Your code is the first 2-3 layers.
- The last 5 layers are very clearly formatting the error and making a safe return of appropriate objects.
- The middle 15 layers are classes and methods of `dplyr`, `rlang`, `purrr` and `base` and maybe packages you have never heard of, passing very opaque things to one another, your informatively named objects are abstracted somewhere in `...`, `.x` and `envir...`
- ... and you are still left wondering what's going on.



rlang::global_handle() in .Rprofile makes traceback better

Error:

! Oops!

▼ [Hide Traceback](#) [Fix](#) [Explain](#)

```
1. | global f(j())
2. |   global g(a)
3. |     global h(b)
4. |       global i(c)
5. | global j()
6. |   global k()
```



interactive debugger



browser()

```
1 f <- function(x) {  
2   browser()  
3   x + 1  
4 }  
5  
6 g <- function(x) f(x)
```

`browser()` opens the interactive debugger.

1. Modify the function by inserting a `browser()` statement.
2. Source the function.
3. Execute the function.



Interactive debugger

The screenshot shows the RStudio IDE interface during a debugging session. The editor window at the top left contains the following R code in `my_functions.R`:

```
1 f <- function(x) {
2   browser()
3   x + 1
4 }
5
6 g <- function(x) f(x)
```

The console at the bottom left shows the following commands and output:

```
> source("demo/my_functions.R")
> g("a")
Called from: f(x)
Browse[1]> |
```

The Environment pane on the right shows the current environment with a variable `x`. The Traceback pane shows the sequence of function calls:

```
f(x) at my_functions.R:2
g("a") at my_functions.R:6
```

The bottom of the RStudio window features a toolbar with buttons for Files, Plots, Packages, Help, Viewer, and Presentation.



Interactive debugger tips

Investigate objects

`ls()`, `ls.str()`,
`str()`, `print()`

Control execution

command	operation
<code>n</code>	next statement
<code>c</code>	continue (leave interactive debugging)
<code>s</code>	step into function call
<code>f</code>	finish loop / function
<code>where</code>	show previous calls
<code>Q</code>	quit debugger



Use `browser()` to set a break point in code

```
g <- function(b) {  
  browser()  
  h(b)  
}  
f(10)
```

[Copy](#)

browser() can be conditional

```
g <- function(b) {  
  if (b < 0) {  
    browser()  
  }  
  h(b)  
}
```

[Copy](#)

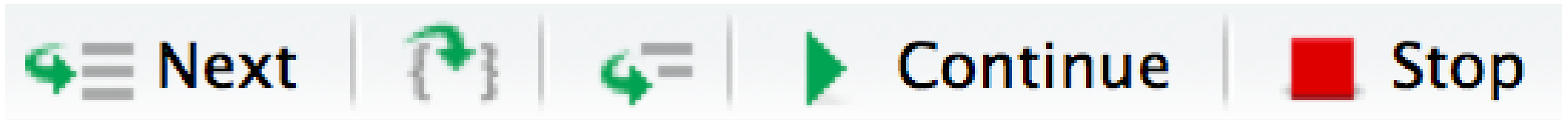
A better practice would be to define your own debugging flags that would be turned off or disappear in production code

```
if (exists("my_debugging_flag")) browser()
```



browser() provides special commands

These commands work in the Console; RStudio also makes the toolbar buttons available.



- Next (**n**): Execute the next step
- Step into (**s**): Dive into function (or **n**)
- Finish (**f**): Finish execution of the current loop/function
- Continue (**c**): Continue regular execution of the function (leave interactive)
- Stop (**Q**): Stop debugging, terminate the function, and return to the global workspace



browser() practical tips

- RStudio runs its best attempt to show the code that is being executed in the “Source” pane
 - parsed [AST](#) for external functions (no comments, no indents);
 - actual source files within your packages when you are in the package project.
- That means you can select the code and [Ctrl+Enter](#) to execute it
 - if in doubt about the source of the error, do that rather than hitting [n](#) “Next”
 - especially if the computations leading to this point in your code are costly in terms of time



`debug(fn_name)` to insert `browser()` in first line of `fn_name()`

- `undebbug(fn_name)` to remove it
- `debugonce(fn_name)` to do it once (similar to rerun with debug)
- `utils::setBreakpoint("file_name", line_number)`



Call stacks printed by **traceback()**,
browser() & **where**, and **recover()** are not
consistent



RStudio has tools for debugging

- Click “Rerun with Debug” in error message
- Enable `Debug > On Error > Break in Code` to always jump to error



Set breakpoints in RStudio for virtual `browser()`

Activate:

- Click to left of line number, or
- Press **Shift + F9**

Downsides:

- Fail in [unusual situations](#)
- Can't be conditional



Use `options(error = recover)` for interactive debugging prompt

```
options(error = recover)
f("x")
#> Error: `d` must be numeric
#>
#> Enter a frame number, or 0 to exit
#>
#> 1: f("x")
#> 2: debugging.R#1: g(a)
#> 3: debugging.R#2: h(b)
#> 4: debugging.R#3: i(c)
#>
#> Selection:
```

[Copy](#)

Turn off with `options(error = NULL)`



Q your code vs  their code



Debugging your code

	 traceback	 interactive debugger	 your code	 their code
Functions		<code>print() / cat() / message()</code>	output diagnostic information in code	
	  	<code>traceback()</code>	locate error	
	 	<code>browser()</code>	open interactive debugger at browser() location	
	  	<code>debug() / debugonce()</code>	automatically open interactive debugger when function called	
	  	<code>trace()</code>	start debugger at specific location in function	
options()	  	<code>error = rlang::entrace</code>	provides richer traceback & error handling	
	  	<code>error = recover</code>	select frame from traceback to enter interactive debugger	
	 	<code>warn = 2</code>	upgrade warnings to errors	
	  	<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace	
IDE	 	breakpoint	open interactive debugger at breakpoint location	
	  	Debug -> On Error -> Error Inspector	provides "Show Traceback" & "Rerun with Debug" console options	
	 	Debug -> On Error -> Break in code	open interactive debugger on error	



Your turn, exercise 01

Download the exercises

- with `usethis::use_course("rstats-wtf/wtf-debugging")`, or
- download the zip folder from <https://github.com/rstats-wtf/wtf-debugging>

Complete 01_exercise to practice debugging your own code.

- `01_debugging_spartan.R` (directions to explore without suggested code)
- `01_debugging_comfy.R` (directions to explore with suggested code)
- `01_debugging_solution.R` (directions to explore with code solutions)



Debugging in RStudio

 **traceback**

 **interactive debugger**

 **your code**

 **their code**

Functions

	<code>print() / cat() / message()</code>	output diagnostic information in code
  	<code>traceback()</code>	locate error
 	<code>browser()</code>	open interactive debugger at <code>browser()</code> location
  	<code>debug() / debugonce()</code>	automatically open interactive debugger when function called
  	<code>trace()</code>	start debugger at specific location in function

options()

  	<code>error = rlang::entrace</code>	provides richer traceback & error handling
  	<code>error = recover</code>	select frame from traceback to enter interactive debugger
 	<code>warn = 2</code>	upgrade warnings to errors
  	<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace

IDE

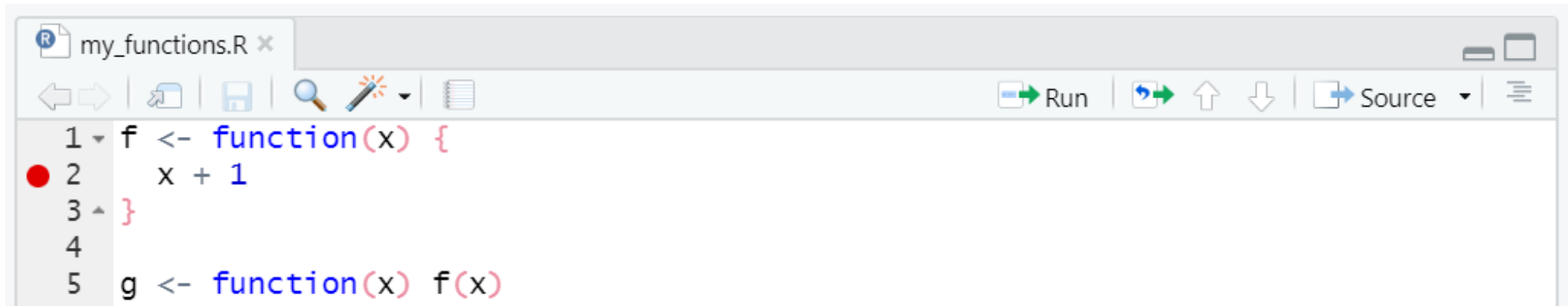
 	breakpoint	open interactive debugger at breakpoint location
  	Debug -> On Error -> Error Inspector	provides "Show Traceback" & "Rerun with Debug" console options
 	Debug -> On Error -> Break in code	open interactive debugger on error



Editor breakpoints

red circle = breakpoint = `browser()`

(but you don't have to change your code)



The screenshot shows an R editor window titled 'my_functions.R'. The code is as follows:

```
1 f <- function(x) {  
2   x + 1  
3 }  
4  
5 g <- function(x) f(x)
```

A red circle breakpoint is set on line 2. The editor includes a toolbar with icons for navigation, search, and execution, along with 'Run' and 'Source' buttons.

Set / re-set an editor breakpoint:

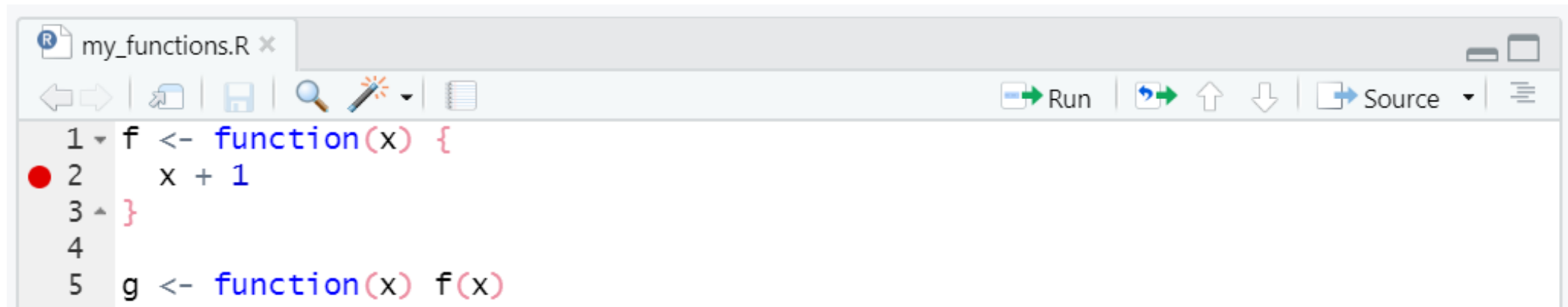
- click to the left of the line number in the source file
- press Shift+F9 with your cursor on the line



Editor breakpoints

red circle = breakpoint = `browser()`

(but you don't have to change your code)



The screenshot shows an R IDE editor window titled 'my_functions.R'. The editor contains the following R code:

```
1 f <- function(x) {  
2   x + 1  
3 }  
4  
5 g <- function(x) f(x)
```

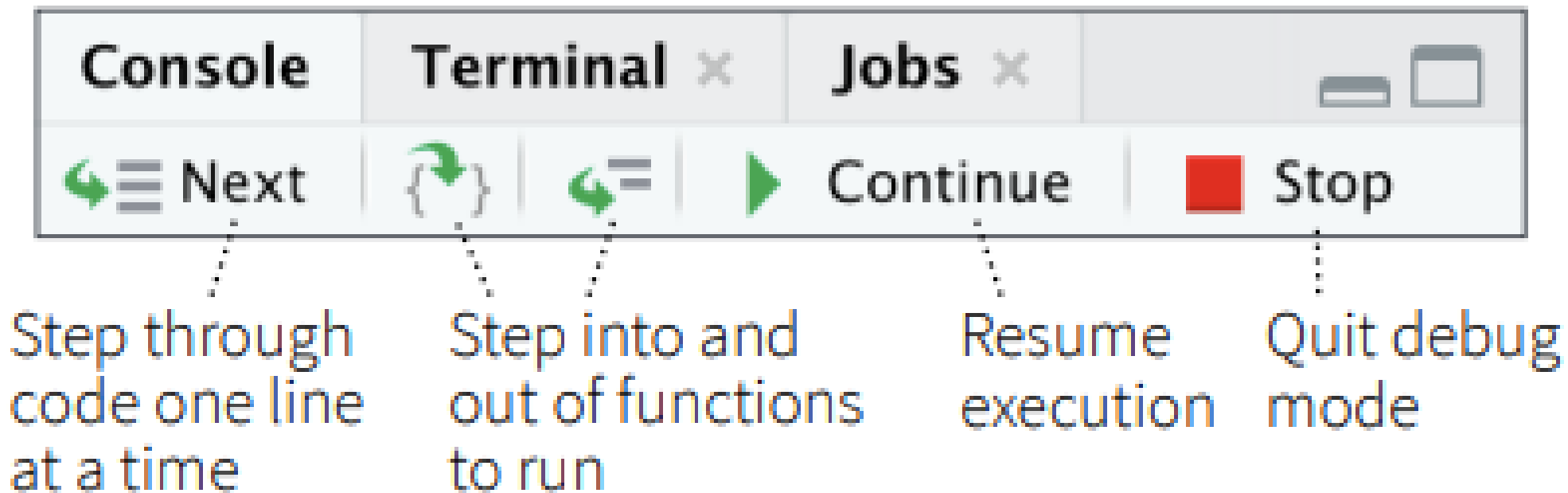
A red circle breakpoint is placed on line 2. The toolbar at the top includes buttons for navigation, search, and execution. The 'Run' button is highlighted, and the 'Source' button is also visible.

To activate, either

- click IDE Source button, or
- `debugSource("demo/my_functions.R")`

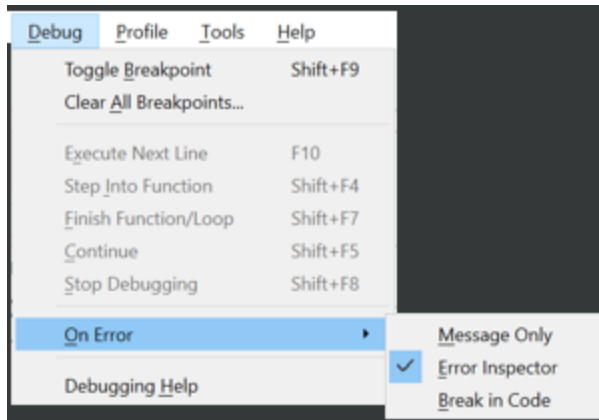


Debugging console



IDE on error

Automatically invoke actions on error.



New set up for demonstration.

```
1 f <- function(x) { strsplit(x, ",") }
2 g <- function(x) { f(x) }
3 g("a,b")
```

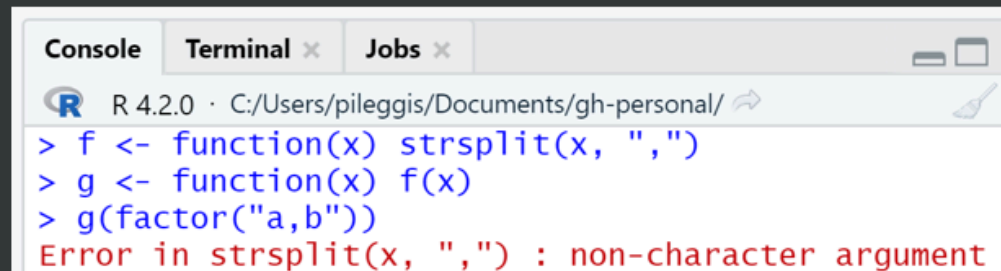
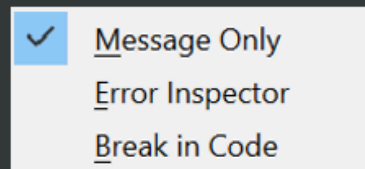
```
[[1]]
[1] "a" "b"
```

```
1 g(factor("a,b"))
```

```
Error in `strsplit()`:
! non-character argument
```



IDE message only

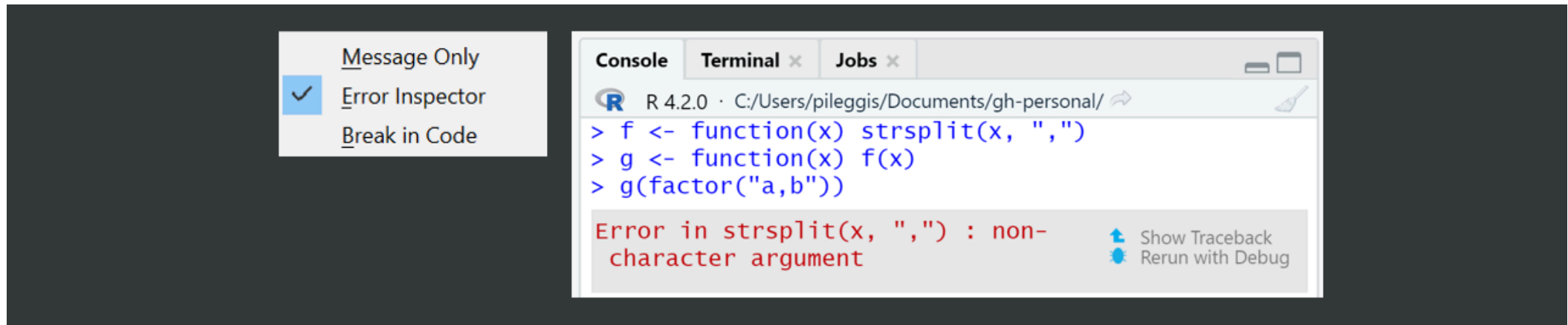


A screenshot of an R console window. The window title bar shows 'Console', 'Terminal x', and 'Jobs x'. The console content shows the following R code and error message:

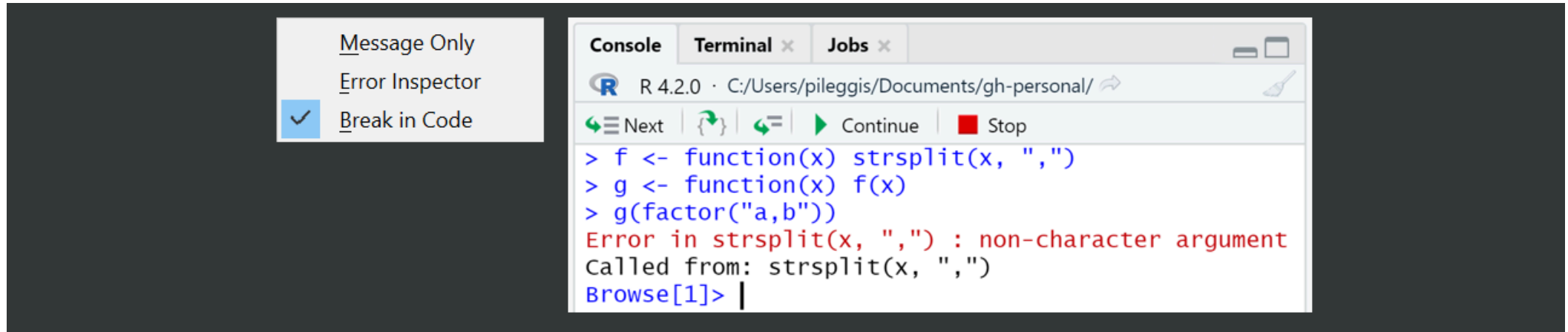
```
R 4.2.0 · C:/Users/pileggis/Documents/gh-personal/
> f <- function(x) strsplit(x, ",")
> g <- function(x) f(x)
> g(factor("a,b"))
Error in strsplit(x, ",") : non-character argument
```



IDE error inspector



IDE break in code



Error inspector

```
1 # IDE Error Inspector not triggered
2 f <- function(x) x + 1
3 g <- function(x) f(x)
4 g("a")
5
6 # IDE Error Inspector not triggered
7 strsplit(factor("a,b"), ",")
8
9 # IDE Error Inspector not triggered
10 f <- function(x) strsplit(x, ",")
11 f(factor("a,b"))
12
13 # IDE Error Inspector yes triggered
14 g <- function(x) f(x)
15 g(factor("a,b"))
```

The error inspector is only invoked if **your code** is involved.



Debugging in the RStudio IDE

 **traceback**

 **interactive debugger**

 **your code**

 **their code**

Functions

	<code>print() / cat() / message()</code>	output diagnostic information in code
  	<code>traceback()</code>	locate error
 	<code>browser()</code>	open interactive debugger at <code>browser()</code> location
  	<code>debug() / debugonce()</code>	automatically open interactive debugger when function called
  	<code>trace()</code>	start debugger at specific location in function

options()

  	<code>error = rlang::entrace</code>	provides richer traceback & error handling
  	<code>error = recover</code>	select frame from traceback to enter interactive debugger
 	<code>warn = 2</code>	upgrade warnings to errors
  	<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace

IDE

 	breakpoint	open interactive debugger at breakpoint location
  	Debug -> On Error -> Error Inspector	provides "Show Traceback" & "Rerun with Debug" console options
 	Debug -> On Error -> Break in code	open interactive debugger on error



Your turn, exercise 02

Complete 02_exercise to practice debugging a slightly different version of your own code, using features from the RStudio IDE.



Debugging their code

	 traceback	 interactive debugger	 your code	 their code
Functions			<code>print() / cat() / message()</code>	output diagnostic information in code
	  		<code>traceback()</code>	locate error
	 		<code>browser()</code>	open interactive debugger at <code>browser()</code> location
	  		<code>debug() / debugonce()</code>	automatically open interactive debugger when function called
	  		<code>trace()</code>	start debugger at specific location in function
options()	  		<code>error = rlang::entrace</code>	provides richer traceback & error handling
	  		<code>error = recover</code>	select frame from traceback to enter interactive debugger
	 		<code>warn = 2</code>	upgrade warnings to errors
	  		<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace
IDE	 		breakpoint	open interactive debugger at breakpoint location
	  		Debug -> On Error -> Error Inspector	provides "Show Traceback" & "Rerun with Debug" console options
	 		Debug -> On Error -> Break in code	open interactive debugger on error



debug()

`debug()` = `browser()` / `breakpoint` in first line of function

```
1 # source if code is yours, otherwise not necessary
2 source("demo/my_functions.R")
3 # set debugging flag
4 debug("g")
5 g("a")
6 # turn off debugging flag
7 undebug("g")
```

- interactive debugger is initiated every time `g()` is executed, until `undebug("g")`
- depending on function internals, this can... trap you in the debugger 😬



debugonce()

`debugonce()` = `browser()` / `breakpoint` in first line of function

```
1 # source if code is yours, otherwise not necessary
2 source("demo/my_functions.R")
3 # set debugging flag
4 debugonce("g")
5 g("a")
```

- one time only!
- interactive debugger initiated a **single** time when `g()` is executed



options(error = recover)

✓ we already discussed

`options(error = rlang::entrace)`

for a richer traceback on error

🤖 `options(error = recover)`

1. displays an interactive prompt with frames
2. you select the frame to enter the debugger



recover example

```
1 source("demo/my_functions.R")
2 options(error = recover)
3 g("a")
4 options(error = NULL)
```

```
> options(error = recover)
```

```
> g("a")
```

```
Error in x + 1 : non-numeric argument to binary operator
```

```
Enter a frame number, or 0 to exit
```

```
1: g("a")
```

```
2: #1: f(x)
```

```
selection: |
```



trace() overview

`trace(what = fun, tracer = browser)`

is equivalent to

- inserting `browser()` in first line of function
- `debug(fun)`

you can also insert *any* code at *any* location in function

- `trace(what = fun, tracer = browser, at = 2)`
inserts `browser()` at second step of `fun`

`untrace(fun)` cancels the tracing



trace() without a specified step

```
1 trace(what = colSums, tracer = browser)
2 colSums(1:3)
3 # ls.str() in browser
4 untrace(colSums)
```

`trace(what = colSums, tracer = browser)`
is equivalent to

- inserting `browser()` in first line of `colSums`
if we had the source code
- `debug(colSums)`



navigating function steps

investigate 🤔 the function with `as.list()` + `body()`

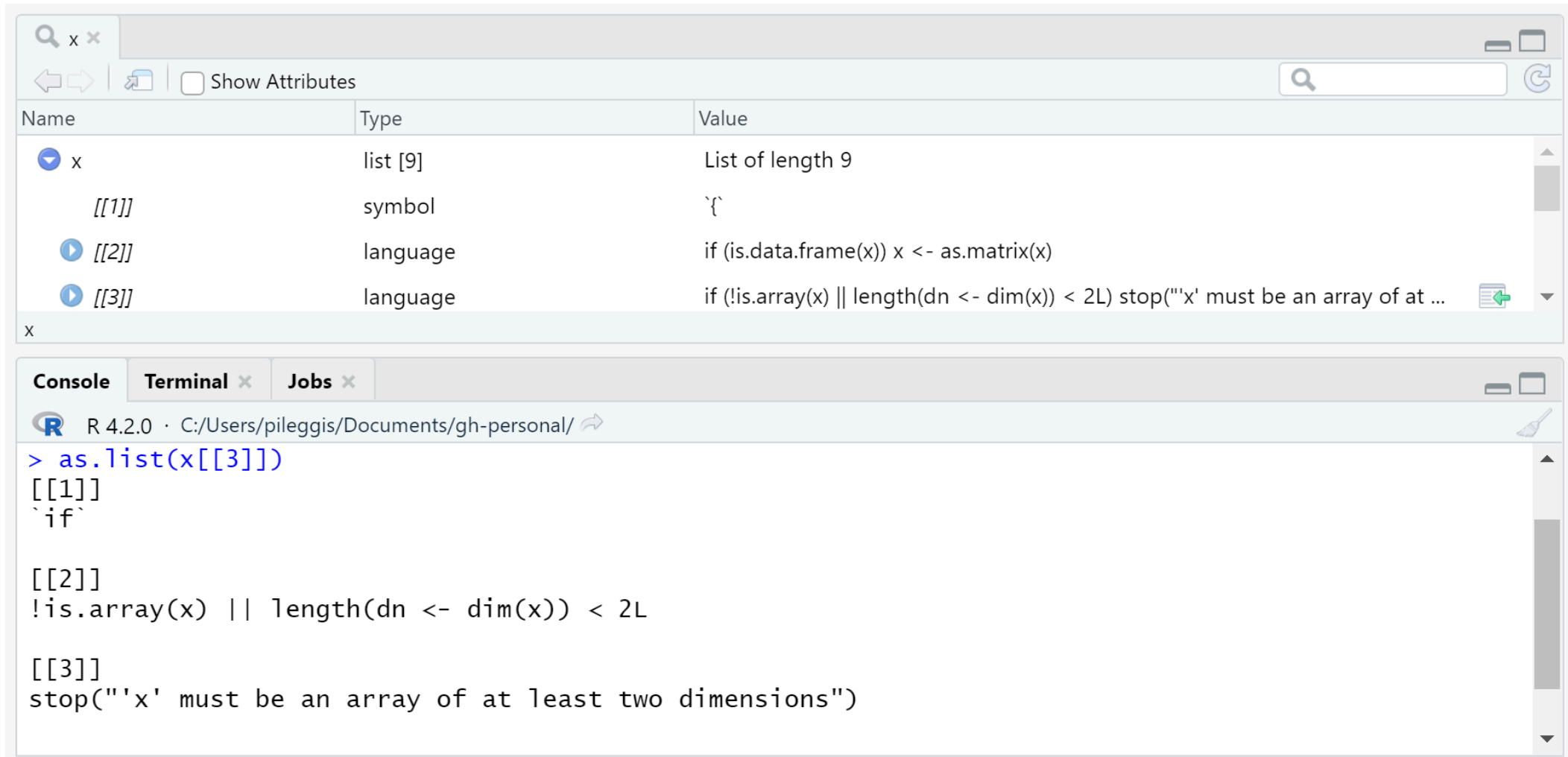
```
1 x <- as.list(body(colSums))
2 View(x)
```

```
> colSums
function (x, na.rm = FALSE, dims = 1L)
{
  if (is.data.frame(x))
    x <- as.matrix(x)
  if (!is.array(x) || length(dn <- dim(x)) < 2L)
    stop("'x' must be an array of at least two dimensions")
  if (dims < 1L || dims > length(dn) - 1L)
    stop("invalid 'dims'")
  n <- prod(dn[id <- seq_len(dims)])
  dn <- dn[-id]
  z <- if (is.complex(x))
    .Internal(colSums(Re(x), n, prod(dn), na.rm)) + (0+1i) *
    .Internal(colSums(Im(x), n, prod(dn), na.rm))
  else .Internal(colSums(x, n, prod(dn), na.rm))
  if (length(dn) > 1L) {
    dim(z) <- dn
    dimnames(z) <- dimnames(x)[-id]
  }
  else names(z) <- dimnames(x)[[dims + 1L]]
  z
}
<bytecode: 0x000001b47ed54c48>
<environment: namespace:base>
```

Name	Type	Value
▼ x	list [9]	List of length 9
[[1]]	symbol	`{`
▶ [[2]]	language	if (is.data.frame(x)) x <- as.matrix(x)
▶ [[3]]	language	if (!is.array(x) length(dn <- dim(x)) < 2L) stop("'x' must be an array of at ...
▶ [[4]]	language	if (dims < 1L dims > length(dn) - 1L) stop("invalid 'dims'")
▶ [[5]]	language	n <- prod(dn[id <- seq_len(dims)])
▶ [[6]]	language	dn <- dn[-id]
▶ [[7]]	language	z <- if (is.complex(x)) .Internal(colSums(Re(x), n, prod(dn), na.rm)) + (0+ ...
▶ [[8]]	language	if (length(dn) > 1L) { dim(z) <- dn dimnames(z) <- dimnames(x)[-id] } el ...
[[9]]	symbol	`z`



identify function step



The screenshot displays the R Studio interface. The top pane shows the environment with a list object 'x' containing 9 elements. The bottom pane shows the console output of the command `as.list(x[[3]])`.

Name	Type	Value
x	list [9]	List of length 9
[[1]]	symbol	`{`
[[2]]	language	if (is.data.frame(x)) x <- as.matrix(x)
[[3]]	language	if (!is.array(x) length(dn <- dim(x)) < 2L) stop("'x' must be an array of at ...")


```

> as.list(x[[3]])
[[1]]
`if`

[[2]]
!is.array(x) || length(dn <- dim(x)) < 2L

[[3]]
stop("'x' must be an array of at least two dimensions")
  
```



trace() at specified step

```
1 # identify spot to insert code
2 as.list(x[[3]][[1]])
3 # equivalent notation
4 as.list(x[[c(3, 1)]])
5
6 # insert browser at step 3.1
7 trace(colSums, browser, at = list(c(3, 1)))
8 # execute function
9 colSums(1:3)
10 # ls.str() handy in browser
11 # cancel tracing
12 untrace(colSums)
```



Debugging their code

	 traceback	 interactive debugger	 your code	 their code
Functions			<code>print() / cat() / message()</code>	output diagnostic information in code
		 	<code>traceback()</code>	locate error
			<code>browser()</code>	open interactive debugger at browser() location
		 	<code>debug() / debugonce()</code>	automatically open interactive debugger when function called
		 	<code>trace()</code>	start debugger at specific location in function
options()		 	<code>error = rlang::entrace</code>	provides richer traceback & error handling
		 	<code>error = recover</code>	select frame from traceback to enter interactive debugger
		 	<code>warn = 2</code>	upgrade warnings to errors
		 	<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace
IDE			<code>breakpoint</code>	open interactive debugger at breakpoint location
	 		<code>Debug -> On Error -> Error Inspector</code>	provides "Show Traceback" & "Rerun with Debug" console options
			<code>Debug -> On Error -> Break in code</code>	open interactive debugger on error



Your turn, exercise 03

```
1 # already installed in the Posit Cloud work space  
2 # install.packages("devtools")  
3 # devtools::install_github("rstats-wtf/wtfdbg")
```

Complete 03_exercise to practice debugging others' code.



Advanced topics



Special cases



Quarto/Rmarkdown

Troubleshooting:

1. rmarkdown chunk option `error = TRUE` enables knitting with errors
2. insert `knitr::knit_exit()` and interactively work through .Rmd

Debugging:

1. Adv R Ch 22.5.3 RMarkdown <https://adv-r.hadley.nz/debugging.html#rmarkdown>
2. WTF Ch 11.4 Debugging in Rmarkdown documents <https://rstats.wtf/debugging-r-code.html#debugging-in-r-markdown-documents>



Piped expressions

tracebacks can be verbose with pipes

```
1 options(error = rlang::entrace, rlang_backtrace_on_error = "branch")
```

gives trimmed tracebacks when using pipes

- more [contributed answers on RStudio Community](#)
- Matt Dray 2019 blog post [Fix leaky pipes in R](#)



Warnings

If you want to dig deeper into a warning, you can convert them to errors to initiate debugging tools.

```
1 ?options
2 options(warn = 0) # default, stores warnings until top-level function returns
3 options(warn = 1) # warnings are printed as they occur
4 options(warn = 2) # upgrades warnings to errors
5
6
7 # initiate recover on warning
8 options(warn = 2, error = recover)
9 # restore original settings
10 options(warn = 0, error = NULL)
```



Debugging special cases

	 traceback	 interactive debugger	 your code	 their code
Functions			<code>print() / cat() / message()</code>	output diagnostic information in code
		 	<code>traceback()</code>	locate error
			<code>browser()</code>	open interactive debugger at <code>browser()</code> location
		 	<code>debug() / debugonce()</code>	automatically open interactive debugger when function called
		 	<code>trace()</code>	start debugger at specific location in function
options()		 	<code>error = rlang::entrace</code>	provides richer traceback & error handling
		 	<code>error = recover</code>	select frame from traceback to enter interactive debugger
		 	<code>warn = 2</code>	upgrade warnings to errors
		 	<code>rlang_backtrace_on_error = "branch"</code>	displays simplified backtrace
IDE			breakpoint	open interactive debugger at breakpoint location
		 	Debug -> On Error -> Error Inspector	provides "Show Traceback" & "Rerun with Debug" console options
			Debug -> On Error -> Break in code	open interactive debugger on error



Non-interactive debugging



Use `callr::r()` or fresh start to look for differences

`callr::r(f, list(1, 2))` calls `f(1, 2)` in a fresh session

- Global env
- Packages
- Object
- Working directory
- `PATH` environment variable
- `R_LIBS` environment variable



dump.frames() is the equivalent to **recover()** for non-interactive code.

```
# In batch R process ----
dump_and_quit <- function() {
  # Save debugging info to file last.dump.rda
  dump.frames(to.file = TRUE)
  # Quit R with error status
  q(status = 1)
}
options(error = dump_and_quit)

# In a later interactive session ----
load("last.dump.rda")
debugger()
```

[Copy](#)

Print debugging can help if nothing else works

```
f <- function(a) {  
  cat("f()\n")  
  g(a)  
}  
g <- function(b) {  
  cat("g()\n")  
  cat("b =", b, "\n")  
  h(b)  
}  
h <- function(c) {  
  cat("i()\n")  
  i(c)  
}
```

```
f(10)  
#> f()  
#> g()  
#> b = 10  
#> i()  
#> [1] 20
```

[Copy](#)

Wrap up



Pick your favorite

 your code  their code

A traceback

Q IDE: Debug -> On Error -> Error Inspector

 Q traceback()

 Q options(error = rlang::entrace)

interactive
debugger



Q IDE: Debug -> On Error -> Error Inspector

Q IDE: Debug -> On Error -> Break in code

Q IDE: breakpoint

Q browser()

 Q debug() / debugonce()

 Q trace()

 Q options(error = recover)



Go forth, and learn from your bugs!



 traceback

 interactive debugger

 your code vs
their code



Further reading

Some useful resources on debugging:

- [“Object of type closure is not subsettable”](#) (Jenny Bryan, `rstudio::conf(2020)`)
- [“What They Forgot to Teach You About R” Chapter 12](#) (by Jenny Bryan and Jim Hester)
- [Minimal reprex for a shiny app](#) (video by Hadley Wickham)
- [Debugging in RStudio](#), by Posit
- If you have access to “Code Complete”, it provides [generic debugging principles](#) that are meant to be applicable across programming languages



Additional resources

- WTF Ch 11 *Debugging R code*
<https://rstats.wtf/debugging-r-code.html>
- Advanced R Ch 22 *Debugging*
<https://adv-r.hadley.nz/debugging.html>
- Jenny Bryan 2020 RStudio Conf Keynote *Object of type closure is not subsettable*
<https://github.com/jennybc/debugging#readme>
- Amanda Gadrow 2018 Webinar *Debugging techniques in RStudio*
<https://www.rstudio.com/resources/webinars/debugging-techniques-in-rstudio/>
- Jim Hester 2019 *Introduction to debugging in R and RStudio*
<https://www.jimhester.com/talk/2019-crug-debugging/>
- Maëlle Salmon 2021 *How to become a better R code detective?*
<https://masalmon.eu/2021/07/13/code-detective/>
- Kara Woo 2019 RStudio Conf *Box plots A case study in debugging and perseverance*
<https://www.rstudio.com/resources/rstudioconf-2019/box-plots-a-case-study-in-debugging-and-perseverance/>



References



Advanced: Functions can fail without errors

- Unexpected warning: `options(warn = 2)` turns warnings into errors.
- Unexpected message: Proposed solution in book no longer available.
- Function might never return. Terminate & `traceback()`.
- Crashed R = bug in compiled (C, C++, etc) code.



Advanced: Automated tests help

- Add tests for nearby working code to avoid new bugs.
- Add tests for specific broken cases.
- Add tests for hypotheses.

Concepts of unit tests are generally associated with R package development, see [chapters on testing in “Writing R Packages”](#).

Automated testing sometimes brings a dilemma: you add something to a working function, and the stuff that you added does work according to the new tests, but an old test gets broken. This is may not be an error per se, but it shows a gap in your knowledge of and assumptions about the code (see Norm Matloff’s quote above) and you have to deal with that one way or another.



Advanced: Debugging withing packages

See *Advanced R* book (chapter 8) on classed conditions:

- you can create richer structured objects with

```
rlang::abort(  
  message = "cli-formatted-message",  
  class    = "your-special-class-to-distinguish-from-other-errors",  
  ...  
)
```

- you can pass your situation report in `...` so that error handlers and/or reporters can more effectively peek into your environment that errored
- you can capture the class with `testthat::expect_s3_class()`



Advanced: Automate code style checking

`lintr` can automate checks to warn you if you leave `browser()` statements in your code.

https://lintr.r-lib.org/reference/default_undesirable_functions.html

```
usethis::use_github_action("lint")
```

Live example in devtools <https://github.com/r-lib/devtools/actions/workflows/lint.yaml>



Advanced: Debugging Rmarkdown/Quarto has challenges

- Call `rmarkdown::render("path/to/file.Rmd")` instead of IDE knitting.
 - downside: the content of your current environment propagates into the markdown code
- Use `sink()` for tricky error handling

```
options(error = function() {  
  sink()  
  recover()  
})
```

See also “[Markdown test drive](#)” in Jenny Bryan’s *Happy Git With R* book.



Advanced: RStudio vs Positron

Debugging tools in Positron are still under active development. `browser()` and `debugonce()` are supported, but breakpoints aren't.

- [Positron facilitates debugging C++ code](#) posit::conf(2024), Davis Vaughn, Aug 2024
- [Positron IDE Debugger for R Demo with Nested Functions \(Public Beta\)](#) YouTube, James Balamuta, Jun 2024
- [Positron: current joys and pains](#), blog post, Athanasia Mowinckel, Apr 2025

